

Quasar identification in J-PAS data using simulations from DESI spectra

Andreu Solà and Miguel Antonio Ausejo

Physics engineering students at Polytechnic University of Catalonia

(Dated: May 29, 2025)

The Javalambre Physics of the Accelerating Universe Astrophysical Survey (J-PAS) is able to map a wide field of view of the northern sky, offering unprecedented low-resolution spectrophotometric capabilities. The convolution of the simulated data derived from the Dark Energy Spectroscopic Instrument (DESI), the information scanned with the unique set of over 50 narrow-band filters of J-PAS and the incorporation of realistic noise and observational conditions provides a powerful dataset for quasar classification. In this paper we discuss the performance of different Machine Learning (ML) classification models for quasar classification using J-PAS data using simulations from DESI spectra previously obtained by another group of researchers outside of this work. Moreover, we explain the reasoning behind the use of each classification model as well as the discrepancies in the results obtained.

I. INTRODUCTION

Quasars [1] are astronomical objects located at the center of active galaxies, which emit a large amount of energy due to the absorption of matter and emission of light by supermassive black holes. This phenomenon makes them extremely luminous and visible at very large cosmic distances. Their study provides valuable information about the evolution of the universe and the formation of galaxies. A recent DESI [2] survey published spectra of millions of stars, galaxies and quasars. However, obtaining all this spectra is complicated compared to obtaining photometric images. The J-PAS [3] survey precisely obtains this type of images. The purpose of this experiment is to analyze the J-PAS data and try to observe quasars not previously detected by DESI. Therefore we will train different Machine Learning (ML) classification models with J-PAS data and analyze the results to conclude whether we detected an unidentified quasar. The paper is organized as follows:

In Section 2 we introduce the data used in this work. In Section 3 we explain the pre-processing of this data and all the changes made to optimize its performance. A description of the different classification models used appears in Section 4 and in the last Section 5 we present the results and the discussion of them.

II. DATA

The Javalambre Physics of the Accelerating Universe Astrophysical Survey (J-PAS) is a wide-field photometric survey of the northern sky using a 2.5-meter telescope (JST250) and a system of 56 narrow-band (~ 145 Å) and 3 broad-band filters [4]. This configuration enables low-resolution spectrophotometry ($R \sim 50$) per source, allowing accurate photometric redshift estimation and identification of spectral features like quasar emission lines. J-PAS data processing uses a hybrid machine learning

approach, combining decision trees for feature selection with convolutional neural networks (CNNs) [5] for classification using full spectral energy distributions (SEDs) [6]. Training uses mock catalogs and features such as flux ratios, broad-band colors, and morphological parameters. This enables accurate object classification and photometric redshift estimation with precision $\sigma_n \simeq 0.007 \cdot (1+z)$.

J-PAS mainly provides photometric data, but not full detailed spectra. Spectral data from Dark Energy Spectroscopic Instrument (DESI) can complement the previously mentioned photometric data in order to obtain a much more complete dataset. This strategy is used to simulate what the objects observed with spectroscopy would look like through J-PAS narrow-band filters, and then develop methods to recognize quasars with this new simulated data. In this way, possible errors related to the limitations of just using photometric data can be avoided, improving the accuracy and precision of the quasar identification process.

The structure of the data used in this work, provided by our supervising professor, is a combination of both real and simulated data. It consists of spectra obtained via DESI, convoluted with the values of the fluxes for different filters obtained via J-PAS and some added realistic noise. Its large volume and combination of both photometric and spectral data makes it particularly well-suited for training machine learning models focused on quasar detection and classification.

The dataset contains a file with photometric flux measurements across multiple filters, along with their associated noise, stored in NumPy array (.npz) format, and also a catalog (.csv) listing key object attributes, including coordinates (RA, DEC), morphological (MORPHTYPE) and spectral (SPECTYPE) classifications, and spectroscopic redshift (REDSHIFT).

In our machine learning models, fluxes act as input features while SPECTYPE serves as the true labels for the classification tasks. REDSHIFT would not be available with just J-PAS observations, so even though the

information it provides can be really useful for the training, we must consider the limitation it might imply for future uses of the trained model that do not dispose of it. Metadata such as NOISE_SEED, used to generate observational noise, plays a role in robustness testing and validation, aligning with standard practices in mock-based training for large-scale photometric surveys.

III. PRE-PROCESSING

To prepare the data for optimal performance in the ML classification models, we first flattened the original .npz arrays, which had shape $n \times 57 \times 3$, into two-dimensional arrays. Only the second and third columns (fluxes with noise and their associated errors) were retained from the last dimension, as the first column (noise-free flux) was considered unrealistic and irrelevant for training. This processed data was already split into training, validation and testing sets: X_{train} , $X_{\text{validation}}$ and X_{test} , with their corresponding arrays containing the answer of the astronomical object corresponding to that data (y_{train} , etc.) extracted from the ‘SPECTYPE’ column, that could be ‘GALAXY’, ‘STAR’ or ‘QSO’, being the last one associated with a quasar.

In parallel, the associated .csv file containing object properties was loaded into a Pandas DataFrame. From this file, we selected nine relevant columns: RA, DEC, DESI_FLUX_G, DESI_FLUX_R, DESI_FLUX_Z, EBV, MORPHTYPE, NOISE_SEED and NOISE_TILE, for model training. After selecting the desired columns, we concatenated the numerical data with the flux arrays using NumPy.

Since missing values (NaN) can significantly impair the model’s learning process, we systematically identified and removed all rows containing NaNs. Given that these represented a negligible fraction of the dataset, removal was preferred over imputation. This cleaning process required temporarily converting the arrays to DataFrames to take advantage of Pandas’ NaN-handling capabilities, before converting them back to NumPy arrays for training. Moreover, we also standardized the data to equilibrate all the datasets and obtain better results. We used sklearn’s StandardScaler for Python.

IV. ML CLASSIFICATION MODELS

In order to obtain the best result possible, we decided to use different classification models from the Scikit-learn library in python.

1. Random Forest

The first method we have used is the Random Forest [7]. It is an ensemble learning algorithm for classification and regression. It builds multiple decorrelated decision

trees by training each on a random subset of data (bootstrapping) and considering a random subset of features at each split. The final prediction is the aggregate of individual tree outputs (majority vote or average). For instance, a random forest may combine the outputs of several decision trees, each trained on a random subset of the data, to classify an object as a quasar, star or galaxy based on its fluxes and physical properties.

Random Forest’s inherent randomness reduces overfitting, strengthens accuracy and robustness and allows the algorithm to effectively work with large amounts of data. Random Forests are recognized as versatile and widely used in machine learning.

2. K-Nearest Neighbors

The K-Nearest Neighbors (KNN) [8] algorithm is a non-parametric supervised learning method used for classification and regression. It is based on the principle that data points with similar values generally belong to the same class. For a given point, KNN computes its distance to all data points in the training set (typically using Euclidean distance), identifies the k closest ones and gives a class label based on majority voting.

It is important to consider both the need for feature scaling and the $O(n)$ computational cost.

Despite its conceptual simplicity and effectiveness in low-dimensional settings, the KNN algorithm was discarded in our project due to critical limitations when applied to large datasets. Our training data contains approximately 3.5 million points, that is, files of data. This leads to a computationally expensive prediction step for KNN, because it requires determining the distances to all training points ($O(n)$ complexity in each prediction). This leads to significantly long runtimes, making the method impractical for repeated model evaluation and parameter tuning.

One possible solution would be to reduce the dataset size by selecting only the most representative samples through a Clustering algorithm. However, that would require considerable additional work that would be outside the project’s range and schedule. Additionally, since our analysis involves evaluating feature importance using permutation-based methods, the overhead of performing repeated predictions with KNN becomes even more substantial, further justifying its exclusion.

Overall, the resource demands of KNN, both in terms of memory and computation time, made it unsuitable for our specific dataset and objectives, especially considering the more scalable and efficient alternatives available for classification tasks in high-dimensional spaces.

3. Stochastic Gradient Descent

Stochastic Gradient Descent (SGD, [9]) is a variation of the usual gradient descent algorithm. It has several

advantages in terms of efficiency and scalability compare to the traditional version, making it the more desirable method for deep-learning related projects.

The traditional Gradient Descent (GD) [10] is an iterative algorithm with the goal of minimizing the loss function that represents how different the model's predictions are from the actual values. To minimize this loss function, the algorithm adjusts the parameters to achieve the smaller error possible.

For large datasets, computing the gradient included in the loss function using all data points can be slow and memory-intensive [11]. This is where the variation of SGD over the traditional model plays a key role. Instead of using all the extension of the dataset to compute the gradient at each step (unsuitable for our project as we have seen for KNN), SGD uses only one random data point at each iteration. This allows for the obtention of lower computation time, which is a feature to take into account due to our limited time-span to make all the computational work.

4. Multilayer Perceptron (MLP)

A Multilayer Perceptron (MLP) is a type of feedforward artificial neural networks characterized by multiple layers of interconnected neurons, designed to model complex non-linear relationships. Its architecture is based on an input layer, hidden layers and an output layer. The interconnections are weighted, that is, each connection has a weight representing the importance of that connection [12]. Each neuron computes a transformation of its inputs with its weights and biases and an activation function (we used ReLU). These nonlinearities enable MLPs to approximate complex functions. Optimization algorithms, like Adam, are used to adjust weights efficiently [13]. The network learns through forward propagation (computing predictions), loss computation (measuring error) and backpropagation (computing gradients to update weights) [14]. MLPs are widely applied particularly when the task involves capturing non-linear dependencies in data, such as image classification.

V. RESULTS AND DISCUSSION

This section explains the data modifications that were made to achieve the best results for each classification model. It is important to note that we based the performance of our classification models via three different metrics: precision, recall and f1-score. All of them range from values 0 to 1, being 1 the best possible outcome.

Precision [15] focuses in quantifying the accuracy of positive predictions made by a model. In our case, we want a good precision to only predict as correct the real quasars and avoid mistaking galaxies or stars as quasars. It can be thought as the number of true positives over the total of predicted positives, both correct and incorrect

ones.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

Recall [16] measures to which extent the classification model is able to identify all members of a given class. In our case, we must achieve a good recall in order to avoid missing quasars during the identification process. The equation for recall consists in evaluating the number of true positives over both the true positives and the false negatives detected by the algorithm.

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

Finally, the F1-score [17] evaluates the accuracy of a classification model. It is the conjunction between precision and recall, providing a balance between the two. That is the reason why f1-score is overall the most important parameter to optimize, as it includes both precision and recall.

Then we optimized the parameters for each classification model. As we stated before, the K-Nearest Neighbors model was ruled out and therefore there were only 3 models left: the Random Forest (RF), the Stochastic Gradient Descent (SGD) and the Multilayer Perceptron (MLP).

For both the RF and SGD we started using the GridSearchCV [18] technique from the scikit-learn library, which allows to obtain the best combination of hyperparameters in a systematic way. However, due to the high computational cost and time requirements and the decent but not quite optimal results, other alternatives for parameter tuning were preferred. In particular, the Optuna [19] optimization framework obtained better results than GridSearchCV and in a considerable smaller amount of time, reason why it was selected as the definitive technique. Regarding MLP, the RandomizedSearchCV was preferred over the alternatives due to its better performance in optimizing continuous parameters that can take a wide range of values such as those present in MLPs (learning rate for instance). Once the optimized parameters were found, we could evaluate the best performance of each model, obtaining the results shown in Table 1:

Model	Precision	Recall	F1-Score
RF	0.7147	0.7825	0.7470
SGD	0.4332	0.6811	0.5296
MLP	0.8241	0.6010	0.6950

TABLE I. Performance of each classification model.

It is clear that RF is the best performing model followed by MLP, leaving SGD as the worst of the three. Since SGD updates the model parameters using only a single data point per iteration to minimize the loss function, it is particularly sensitive to points with contradictory labels. In our case, as we checked, this often occurs

between galaxies and quasars, which share similar photometric features and can sometimes be easily confused. For example, when observing a near quasar, both the active nucleus and the surrounding galactic disk can be seen. This means a single object may show properties typical of both classes, potentially leading the model to interpret it as two distinct objects. This ambiguity introduces confusion during parameter updates: gradient directions can become contradictory and unstable. As a result, the model’s convergence is negatively affected, and the loss surface may have an ill-defined descent direction, reducing predictive accuracy.

Although this issue impacts SGD more strongly, other methods are also affected since they similarly detect mixed properties in single objects. To address this, an additional photometric feature is needed to better differentiate galaxies from quasars. One possible solution is to use redshift, which helps separate classes by limiting overlapping cases. However, the available redshift data come from spectroscopic observations, which are generally unavailable. Therefore, we need a method to estimate redshift directly from photometric data.

One option is the template fitting technique, which compares observed photometric data to model spectra of galaxies and quasars at various redshifts and finds the best match to estimate redshift [20]. This approach avoids the need for spectra and can improve classification. Nonetheless, implementing this lies beyond the scope of the present project.

Model	Precision	Recall	F1-Score
RF	0.8894	0.8596	0.8681
SGD	0.8791	0.8512	0.8649
MLP	0.9408	0.8052	0.8677

TABLE II. Performance of each classification model with redshift.

There is a significant improvement for the models trained using the redshift. As stated before, this is due to the fact that the combination of photometric data with the redshift coming from spectroscopic observations allows us to differentiate better each object.

We also evaluated the most important features of the dataset, which correspond to the columns with more relevant information. To do so, we used a feature importances algorithm. We concluded that, for each model, the most important features were:

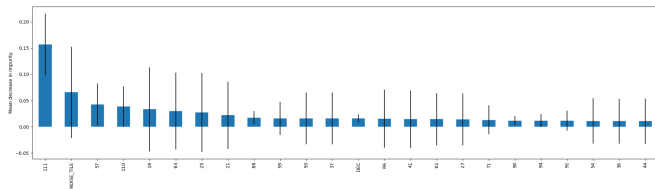


FIG. 1. Random Forest Feature Importances, MDI ≥ 0.02

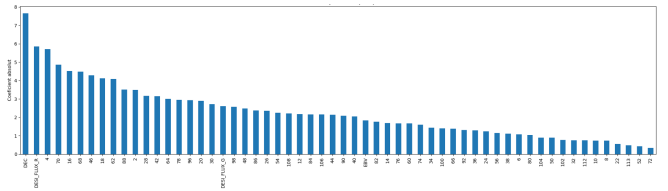


FIG. 2. SGD Feature Importances, Weights ≥ 0.25

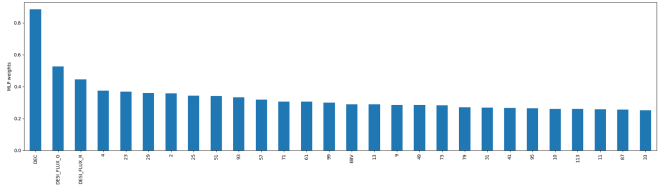


FIG. 3. MLP Feature Importances, Weights ≥ 0.25

As we can see, even though the majority of the most important columns come from the photometric data stored in the .npy file, some features from the .csv file can also be relevant. These include metadata such as noise tile, morphological type, right ascension and declination. These features may carry contextual or observational information. For example, the noise tile reflects the quality of the measurement in different sky regions, ebv traces dust extinction along the line of sight, affecting observed properties and helping to correct or explain variations in the data, and RA and DEC can capture large-scale spatial patterns in the distribution of object types. However, we can also see that some fluxes from .csv file are also very important features for some methods, like SGD and MLP. If we look more in depth, we can see that the most important features of SGD and MLP show a small pattern of similarity, since many of the most important columns are shared by both. This is because the MLP is trained using SGD as an optimization method. Therefore, it is not surprising to see that they share many of these important features.

In conclusion, Random Forest has been the most effective method due to its ability to work with large datasets with multiple different features by using multiple decision trees, reducing overfitting and noise inherent in astrophysical data. Its ability to run tasks simultaneously also improves performance without needing precise hyperparameter tuning. On the other hand, even though MLP is capable of capturing nonlinear relationships, it requires precise parameter tuning and is sensitive to data inconsistencies, especially when trained with SGD, which may diverge due to unstable gradients. SGD, in turn, struggles due to its reliance on individual samples per iteration, amplifying errors in overlapping classes (e.g., quasars vs. galaxies) and the absence of redshift as a key discriminative feature, crucial to disambiguate similar objects.

-
- [1] ESA/Hubble. Quasar, n.d. Accessed: 2025-05-14.
 - [2] David M. Alexander, Tamara M. Davis, I. Perez-Rafols, et al. The desi survey validation: Results from visual inspection of the quasar survey spectra. *The Astronomical Journal*, 165(2):33, 2023.
 - [3] Narciso Benítez, Renato Dupke, Mariano Moles, Laerte Sodré, Javier Cenarro, Antonio Marín-Franch, David Cristóbal-Hornillos, Begoña Ascaso, Jesús Varela, Yolanda Jiménez-Teja, et al. The minijpas survey: A preview of the universe in 56 colors. *Astronomy & Astrophysics*, 653:A31, 2021.
 - [4] S. Bonoli, C. López-Sanjuan, J. Varela, A. J. Cenarro, D. Cristóbal-Hornillos, et al. The minijpas survey: A preview of the universe in 56 colors. *Astronomy & Astrophysics*, 653:A31, 2021.
 - [5] Gaurav Sharma, Pradeep Kumar, and R. C. Tripathi. A survey on deep learning techniques for image classification. *Procedia Computer Science*, 132:1675–1683, 2018.
 - [6] L. Silva, G. L. Granato, A. Bressan, and L. Danese. Modelling the spectral energy distribution of galaxies: introducing the radiative transfer code grasil. *Monthly Notices of the Royal Astronomical Society*, 410(3):2043–2056, 2011.
 - [7] Gérard Biau and Erwan Scornet. A random forest guided tour. *TEST*, 25(2):197–227, 2016.
 - [8] GeeksforGeeks. K-nearest neighbors (knn) algorithm, 2025.
 - [9] Yingjie Tian, Yuqi Zhang, and Haibin Zhang. Recent advances in stochastic gradient descent in deep learning. *Mathematics*, 11(3):682, 2023. Accedido el 7 de mayo de 2025.
 - [10] Moritz Hardt, Ben Recht, and Yoram Singer. Train faster, generalize better: Stability of stochastic gradient descent. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1225–1234, New York, New York, USA, 20–22 Jun 2016. PMLR.
 - [11] Jun Lu. Gradient descent, stochastic optimization, and other tales. *arXiv preprint arXiv:2205.00832*, 2022.
 - [12] Siddique Afraaz N and Maurya Vijayaramachandran. Impact of hidden layer in artificial neural networks. *IOSR Journal of Engineering (IOSRJEN)*, 10(11):33–38, 2020.
 - [13] Siddharth Sharma, Simone Sharma, and Anidhya Athaiya. Activation functions in neural networks. *International Journal of Engineering Applied Sciences and Technology (IJEAST)*, 4(12):310–316, 2020.
 - [14] Timothy P. Lillicrap, Adam Santoro, Luke Marris, Colin J. Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.
 - [15] David M. W. Powers. Evaluation: From precision, recall and f-measure to roc, informedness, markedness & correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.
 - [16] Evidently AI. Accuracy vs. precision vs. recall in machine learning: What’s the difference?, 2025.
 - [17] Reda Yacoubi and Dustin Axman. Probabilistic extension of precision, recall, and f1 score for more thorough evaluation of classification models. In Steffen Eger, Yang Gao, Maxime Peyrard, Wei Zhao, and Eduard Hovy, editors, *Proceedings of the First Workshop on Evaluation and Comparison of NLP Systems*, pages 79–91, Online, November 2020. Association for Computational Linguistics.
 - [18] Great Learning Editorial Team. An introduction to grid-searchcv | what is grid search, September 2024. Accessed: 2025-05-15.
 - [19] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631. ACM, 2019.
 - [20] T. S. R. Babbedge, M. Rowan-Robinson, S. Surace, D. Shupe, F. Fang, C. Lonsdale, D. Farrah, G. Smith, E. A. Gonzalez-Solares, A. Efstathiou, C. Afonso-Luis, M. Polletta, H. Oliver, B. Perez-Fournon, I. Serjeant, L. Jarrett, M. Crockett, and M. Fox. Impz: a new photometric redshift code for galaxies and quasars. *Monthly Notices of the Royal Astronomical Society*, 353(2):654–672, 2004.